Calidad del Software

GUION DE PRÁCTICAS 3

Análisis Estático Integral

---

1. **Duración estimada:** 3 sesiones (6 horas).
2. **Herramientas:** Visual Studio Code, SonarQube for IDE, Gemini, Copilot.

1. INTRODUCCIÓN Y OBJETIVOS

En las prácticas anteriores hemos establecido una línea base (P1) y creado una red de seguridad mediante tests (P2). Ahora estamos listos para limpiar el código con seguridad.

Hasta ahora, veías avisos de **SonarQube** (subrayados amarillos) y quizás pensabas: "*¿Por qué no arreglamos esto?*". La respuesta es: porque sin tests, arreglarlo era peligroso. Ahora que tienes tests, **ha llegado el momento**.

En esta práctica utilizaremos **SonarQube**, apoyándonos en la **Inteligencia Artificial**, para entender y resolver defectos técnicos.

Los objetivos específicos son:

1. **Mejora Continua:** Corregir defectos detectados en la entrega de la Práctica 2.
2. **Uso Pedagógico de la IA:** Aprender a utilizar Copilot/Gemini no para que "escriba código", sino para que nos explique *por qué* un error es peligroso y nos sugiera patrones de corrección.
3. **Mejora de la Calidad:** Resolver los problemas detectados por SonarQube.

2. PREPARACIÓN DEL ENTORNO Y MEJORA CONTINUA

Para comenzar esta práctica, seguiremos un flujo de trabajo iterativo. Continuaremos el proyecto donde lo dejamos, pero asegurando primero que nuestro nuevo entorno está limpio, actualizado y que resolvemos los problemas detectados en la entrega anterior.

2.1. Transición de P2 a P3

Para mantener el histórico de tu trabajo intacto (como una copia de seguridad de lo que entregaste), **no** modifiques la carpeta de la práctica anterior. Sigue estos pasos:

1. Crea una nueva carpeta para esta práctica (ej: `p3A5`).
2. Copia todo el contenido de tu carpeta de la Práctica 2 dentro de la nueva carpeta, **EXCEPTO** los siguientes elementos generados automáticamente:
   - ❌ `build/` ⇒ Contiene binarios y rutas absolutas antiguas que provocarán errores.
   - ❌ `doc/html/` ⇒ La generaremos de nuevo más adelante.
   - ❌ Cualquier archivo ejecutable (`.exe`) que haya quedado suelto en la carpeta `src` o en la raíz (ej: `main.exe`). Los binarios de compilación nunca deben copiarse como parte del código fuente.
3. **Actualiza la configuración de Doxygen:**
   - Localiza el archivo de configuración de Doxygen de la práctica anterior (ej: `p2.Doxyfile`) y **renómbralo** a `p3.Doxyfile`.
   - Abre el fichero con **Doxywizard**.
   - Ve a la pestaña **Wizard** y selecciona la opción **Project**.
   - Modifica el campo **Project Name** para actualizar el título (por ejemplo, cambia "Práctica 2" por "Práctica 3").
   - Guarda los cambios (File ⇒ Save o Ctrl + S) y cierra Doxywizard.
4. **Actualiza la configuración de CMake:**
   - Abre el archivo `CMakeLists.txt` con un editor de texto.
   - Localiza la instrucción `project(Practica2_Tests CXX)`, al principio del fichero, y actualízala con el nuevo nombre `project(Practica3_Tests CXX)`.
   - Guarda los cambios.
5. Abre la carpeta de tu proyecto desde Visual Studio Code.
6. **Reconfiguración inicial:** Al no tener la carpeta `build`, SonarQube y el compilador mostrarán errores (por ejemplo, falta el `compile_commands.json`).
   - Solución: Pulsa el botón **Build** en la barra de estado inferior de VS Code. Esto regenerará la carpeta `build` con las rutas correctas para tu nueva P3.
   - ⚠️ **Atención:** Al hacer esto en una carpeta nueva, VS Code te pedirá en la parte superior que **selecciones un Kit de compilación** (al igual que ocurrió al inicio de la Práctica 2). Asegúrate de seleccionar el compilador instalado en la primera práctica (por ejemplo, `GCC 15.2.0 x86_64-w64-mingw32`).

- Una vez seleccionado el Kit, CMake configurará y regenerará la carpeta `build` desde cero, aplicando el nuevo nombre del proyecto y resolviendo las rutas de tu nueva P3.

2.2. Revisión del Feedback de la Práctica 2

Una vez que tienes la carpeta para esta práctica configurada, con los nombres correctos y compilando perfectamente, es el momento de aplicar la **Mejora Continua**.

1. Revisa detenidamente los comentarios y correcciones recibidos en la Práctica 2.
2. Aplica todas las correcciones necesarias en tu nueva carpeta.

De esta forma, el proyecto de tu Práctica 3 partirá de una base de código corregida y óptima, y estos arreglos ya quedarán incluidos automáticamente en tu próxima entrega final.

3. ANÁLISIS ESTÁTICO INTEGRAL

En la Práctica 2 nos centramos en el **Análisis Dinámico** (Testing), comprobando mediante ejecución que el código hacía lo que debía. En esta práctica abordaremos el **Análisis Estático** mediante el uso de **SonarQube**, lo cual nos permitirá inspeccionar el código fuente *sin ejecutarlo* para encontrar "Code Smells" (señales de alerta en tu código) y deuda técnica: patrones problemáticos, vulnerabilidades y problemas de estilo.

3.1. Configuración del Asistente IA (GitHub Copilot Chat)

Para asistirnos en la resolución de problemas complejos utilizaremos **GitHub Copilot Chat**, un asistente de IA integrado directamente en nuestro entorno de desarrollo.

1. **Activación de la licencia educativa:** Si aún no lo has hecho, debes solicitar el GitHub Student Developer Pack con tu correo UJA para obtener acceso gratuito a Copilot.
2. **Verificación/Instalación en VS Code:** Comprueba si tienes el icono de Copilot en la **esquina inferior derecha** de la barra de estado de VS Code. Si no lo tienes, instala la extensión **GitHub Copilot** (ID: github.copilot-chat).
3. **Inicio de sesión:** Haz clic en el icono de la esquina inferior derecha y selecciona iniciar sesión con tu cuenta de GitHub (la misma donde activaste el paquete de estudiante).

3.2. Corrección de problemas con SonarQube

Los problemas detectados por **SonarQube** son mostrados tanto en la pestaña Problemas, como en otra pestaña llamada **SonarQube**, también visible en el panel inferior de VS Code.

Si seleccionas un problema concreto en la pestaña **SonarQube**, se abrirá una ventana adicional junto a tu código. Esta vista es sumamente útil, ya que SonarQube te explicará de forma detallada por qué ese código es un problema, mostrando ejemplos de código "No conforme" y su equivalente "Conforme":

```
int a = 2.1f; // Noncompliant: loss of floating-point precision

long double f();
double d = 0;
d += f(); // Noncompliant: smaller range of values
```

```
double a = 2.1f; // Compliant: double can represent all floats

long double f();
double d = 0;
d += static_cast<double>(f()); // Compliant: the intent is clear
```

Sigue este ciclo para cada corrección que hagas:

1. **Identificar:** Selecciona un problema.
2. **Resolución:**
   - 🛑 **Falsos Positivos o "Won't Fix":** Como Ingeniero, tú tienes la última palabra, **no SonarQube**. Si la herramienta marca un error que tú consideras que es un "falso positivo" o cuya corrección rompería un requisito innegociable del diseño, no lo corrijas. En su lugar, documenta en la memoria el aviso y **justifica técnicamente por qué has decidido ignorarlo**. Aprender a justificar la omisión de una regla es tan importante como aplicarla.
   - **Manual:** En la mayoría de los casos se trata de problemas fácilmente resolubles. Serás capaz de resolverlos simplemente prestando atención a los ejemplos de código "Conforme" y "No conforme" de SonarQube.
   - **Con IA**: Si el problema te resulta más complejo, puedes hacer uso de la IA:
     - Sitúa el cursor sobre el código marcado con el aviso y Copilot podría sugerirte una solución (`Ctrl+I` para aplicar la corrección propuesta).
     - Revisa el cambio (rojo/verde) y, si tiene sentido, acéptalo.
     - Ten en cuenta que no siempre las soluciones de la IA serán correctas o completas. Por ejemplo, cuando la IA te sugiere aplicar la técnica de "Extracción de Métodos" (*Extract Method*) para reducir la complejidad de determinadas funciones o métodos, la solución propuesta suele centrarse en el cuerpo del fichero `.cpp` donde se ha detectado el problema (la implementación), pero suele ignorar los cambios a aplicar en la interfaz (el fichero `.hpp`). Deberás prestar atención a esto para mantener la consistencia del código y evitar errores de compilación.
3. **Compilar:** Pulsa **Build** para confirmar que no hay errores.
4. **Testear:** Ejecuta tu batería de tests de Google Test.
   - 🟢 **Si pasan en verde:** ¡Enhorabuena! Documenta el cambio para la memoria y pasa al siguiente.
   - 🔴 **Si algún test falla:** Has introducido un bug. **Deshaz el cambio** (Ctrl+Z) y busca otra solución (o pídele a Copilot una alternativa indicándole por qué falló).
   - 💻**Nota**: Esta sección está prevista para cada corrección aplicada. No obstante, si estás resolviendo un conjunto de problemas sencillos similares, puedes aplicar los tres primeros puntos (*identificar*, *resolver* y *compilar*) antes de pasar a la fase de *Testear*. Debes indicarlo explícitamente en la memoria.

3.3. Mantenibilidad: Actualización de la Documentación

Recuerda que una buena documentación es parte indisoluble de la calidad. Asegúrate de que la **documentación Doxygen** esté actualizada (nuevas funciones, modificaciones en parámetros, variables, etc.) y no olvides volver a generarla mediante el uso de **Doxywizard** antes de la entrega final.

4. REQUISITOS DE ENTREGA

Subir al Campus Virtual un archivo **P3X.zip** (ej: P3A5.zip si tu código de grupo es el A5) que contenga:

- **Código Fuente:** Carpeta del proyecto completa **menos** la carpeta `build`.
- **Memoria PDF (P3X.pdf):**
  - **Formato:** Portada, índice, encabezado y pie de página.
  - **Contenido:** Se especifica a continuación.

Contenido de la Memoria (P3X.pdf)

1. **Correcciones P2 (Mejora Continua):** Breve resumen de las mejoras aplicadas al código original tras recibir el feedback de la Práctica 2 (si las hubo).
2. **Auditoría SonarQube:**
   a. Documenta de forma detallada las correcciones realizadas indicando:
   - El aviso original.
   - La solución aplicada (indicando si la resolviste manualmente o usando Copilot).
   - **Breve explicación tuya** de por qué esa solución mejora la calidad del código.
   - ✅**Nota**: puedes agrupar y explicar de forma conjunta aquellos avisos/correcciones que sean similares.

   b. Incluye capturas de pantalla:
   - Para ilustrar las modificaciones realizadas.
   - Para mostrar la correcta ejecución de los tests tras cada corrección o grupo de correcciones aplicadas.

Fecha límite de entrega

La documentación solicitada deberá ser entregada **antes** de la **finalización** de la **última sesión** habilitada para esta práctica (**27/04/2026**).